

HTML



CSS



Modelo de Cajas y Maquetación

Laboratorio de Programación

Ing. Rasjido Jose

UNPA - UACO

AGENDA

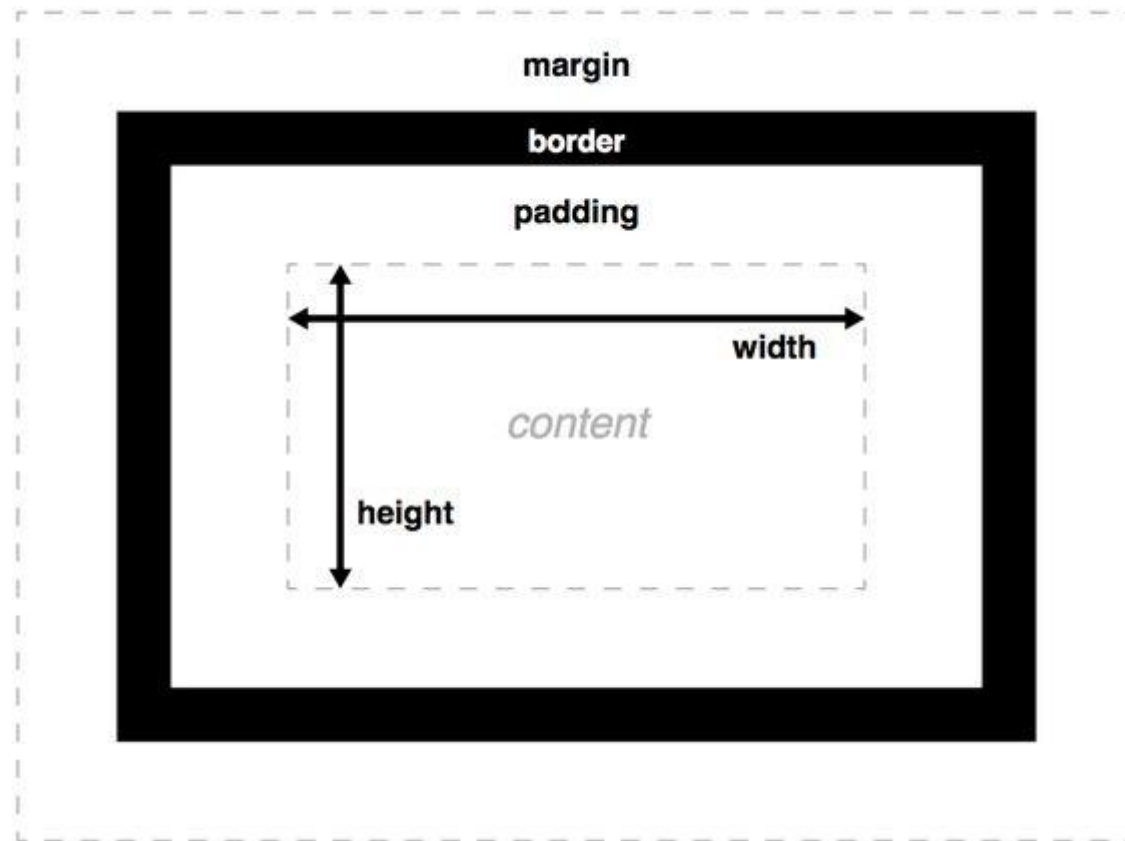
- ✓ **Propiedades de las cajas**
- ✓ Tipos de cajas
- ✓ Esquemas de posicionamiento
- ✓ Flexbox
- ✓ Bibliografía

Propiedades de las cajas

- Cada elemento HTML se presenta como una caja rectangular dentro del diseño del documento.
- El modelo de cajas se visualiza conforme a los estilos establecidos en el CSS. Por lo tanto, las cajas se pueden reducir de tamaño, mover, superponer, etc.
- El modelo de caja determina también la relación de un elemento con respecto a sus adyacente.

Propiedades de las cajas

Alrededor del contenido hay diferentes propiedades que definen una caja o cuadro:



Propiedades de las cajas

➤ Propiedades de una caja

- > **content**. Área de contenido de la caja. Ej: texto, imagen, etc.
- > **width**. Ancho del contenido de la caja.
- > **height**. Alto del contenido de la caja.
- > **padding**. Margen interno entre el borde y el contenido.
- > **border**. Borde o perímetro de la caja.
- > **margin**. Margen externo que rodea la caja y la separa de otras adyacentes.

Propiedades de las cajas

➤ Propiedades de dimensionado – `width` y `height`

Especifican el ancho y alto de una caja.

Valores: `auto` | *`longitud ó porcentaje`* | `min-content` | `max-content` | `fit-content`

No aplica para los elementos de línea.

También se puede especificar un dimensionado mínimo y/o máximo con las propiedades:

`min-width` | `min-height` | `max-width` | `max-height`

Propiedades de las cajas

➤ Márgenes

Los márgenes internos y externos pueden modificarse estableciendo sus valores en el sentido horario:

`margin: top right bottom left`

`padding: top right bottom left`

o individualmente:

`margin-top | margin-right | margin-bottom | margin-left`

`padding-top | padding-right | padding-bottom | padding-left`

Propiedades de las cajas

➤ Bordes

Las propiedades de un borde especifican su grosor, estilo y color. Y son propiedades que aplican para todos los elementos. Forma abreviada:

`border: width style color`

Se puede referenciar el borde de cualquiera de los 4 lados de una caja:

`border-top: width style color`

`border-right: width style color`

`border-bottom: width style color`

`border-left: width style color`

Propiedades de las cajas

➤ Grosor de los bordes - `width`

Puede asumir los siguientes valores:

- > `thin`. Delgado.
- > `medium`. Mediano.
- > `thick`. Grueso.
- > `valor`. Longitud en pixeles.

```
border-width | border-top-width | border-right-width |  
border-bottom-width | border-left-width
```

Propiedades de las cajas

➤ Estilo de los bordes

Puede asumir los siguientes valores:

- > **none** | **hidden**. Sin borde. Se computa como ancho el valor cero.
- > **dotted**. El borde es una serie de puntos.
- > **dashed**. El borde es una serie de líneas cortas.
- > **solid**. El borde es una línea continua.
- > **double**. Dos líneas sólidas.
- > **groove**. El borde aparenta como una ranura o línea hundida.

Propiedades de las cajas

- > **ridge**. Lo contrario a **groove**. La línea aparenta un relieve.
- > **inset**. El borde hace que la caja parezca embebida en el canvas.
- > **ouset**. El borde hace que la caja sobresalga del canvas.

La propiedad puede asumir de uno a cuatro valores:

/ Líneas horizontales solidas y verticales punteadas */*

border-style: **solid dotted;**

border-top-style | border-right-style | border-bottom-style | border-left-style

Propiedades de las cajas

➤ Color de los bordes

Puede asumir los siguientes valores:

> **color**. Ejemplo: #333 | rgba(0 , 255 , 0)

> **transparent**. El borde puede tener un ancho mayor a cero, pero no sería visible.

border-color: #333

border-top-color | border-right-color | border-bottom-color | border-left-color

Propiedades de las cajas

➤ Alcance del dimensionado de la caja - **box-sizing**

Esta propiedad define si las dimensiones de la caja estarán asignadas al contenido o a los bordes de la misma. Puede asumir los valores:

- > **content-box**. Las propiedades **width** y **height** afectan al contenido de la caja. Los bordes, y márgenes internos y externos se calculan aparte y se suman al contenido.
- > **border-box**. Las propiedades de dimensionado incluyen, además del contenido, el borde y margen interno (padding).

Propiedades de las cajas

```
/* Por defecto asume el valor
content-box. En este ejemplo la
caja tendrá una dimensión de
120px hasta su borde. */
```

```
.box {
    box-sizing: content-box;
    width: 100px;
    padding-left: 10px;
    border-left: 10px solid;
}
```

```
/* En este ejemplo la caja tendrá
una dimensión de 100px hasta su
borde.
```

```
El contenido tendrá un ancho de
80px */
```

```
.box {
    box-sizing: border-box;
    width: 100px;
    padding-left: 10px;
    border-left: 10px solid;
}
```

Propiedades de las cajas

➤ Desbordamiento del contenido - **overflow**

Cuando se establecen las dimensiones de una caja, con valores absolutos, el contenido puede desbordarse de su contenedor.

La propiedad **overflow** permite controlar el desbordamiento mediante los valores:

- > **auto**. El desbordamiento se oculta y aparecen barras de desplazamiento.
- > **hidden**. El contenido sobrante no se muestra.
- > **visible**. El contenido sobrante se muestra fuera de la caja.

Propiedades de las cajas

overflow css

visible	hidden	scroll	auto
Caja 1 aprenderaprogra web de didáctica y divulgación de la programación cursos humor y más El tutorial css desde cero	Caja 2 aprenderaprogra web de didáctica y divulgación de la programación cursos humor y más El tutorial css desde cero	Caja 3 aprenderaprogra web de didáctica y divulgación de la programació cursos humor y más El	Caja 4 aprenderaprogra web de didáctica y divulgación de la programació cursos humor y más El

permite
aprender sin
tener
conocimientos
previos

AGENDA

- ✓ Propiedades de las cajas
- ✓ **Tipos de cajas**
- ✓ Esquemas de posicionamiento
- ✓ Flexbox
- ✓ Bibliografía

Tipos de cajas

- CSS contempla dos tipos de cajas, los cuales se corresponden con los elementos de **bloque** y de **línea**.
- En la mayoría de los casos, CSS trabaja de forma similar para ambos tipos.
- Se puede cambiar la fuente, color, fondo y agregar bordes. Sin embargo, cuando se trata de márgenes (internos y externos), los navegadores le dan un tratamiento distinto.

Tipos de cajas

➤ Elementos de bloque

- Cada caja empieza en una línea nueva, y generalmente, ocupa todo el ancho disponible.
- En el flujo de contenido, quedan ordenados uno debajo de otro.
- La distancia respecto a sus adyacentes se puede manejar libremente mediante la propiedad **margin**.
- Pueden contener otros elementos, tanto de línea como de bloque.
- Ejemplos: **div**, **header**, **section**, **nav**, **p**, **h1**, **table**, **ul**, etc.

Tipos de cajas

➤ Elementos de línea

- Cada caja se posiciona a la derecha del último elemento, y solo pasa a la línea siguiente si no hay lugar.
- En el flujo de contenido, quedan ordenados uno al lado del otro.
- No se les puede asignar dimensiones y solo ocupan el espacio necesario para su contenido.
- Pueden contener otros elementos de línea.
- Ejemplos: `a`, `strong`, `span`, `em`, `small`, `img`, etc.

Tipos de cajas

➤ Alterar el tipo de caja - **display**

Es posible cambiar el tipo de un elemento para alterar su comportamiento dentro del modelo de cajas.

La propiedad **display** puede asumir los siguientes valores:

- > **block**. El elemento se presenta como una caja de bloque.
- > **inline**. El elemento se presenta como una caja de línea.
- > **inline-block**. El contenido se formatea como una caja de bloque, pero el elemento en si como una caja de línea.

Tipos de cajas

- > **list-item**. El elemento se formatea como una caja de bloque dentro de una lista.
- > **none**. Provoca que el elemento, y sus descendientes, no se muestre dentro del flujo normal de los elementos.
- > Otros tipos: **table**, **inline-table**, **table-row**, **table-cell**, **table-caption**, **flex**, **grid**, etc.

AGENDA

- ✓ Propiedades de las cajas
- ✓ Tipos de cajas
- ✓ **Esquemas de posicionamiento**
- ✓ Flexbox
- ✓ Bibliografía

Esquemas de posicionamiento

Una caja se puede presentar conforme a tres esquemas de posicionamiento:

- **Flujo normal.** Incluye el formateo de cajas de bloque y de línea, y el posicionamiento relativo de las cajas.
- **Flotante.** En este modelo los elementos son sacados del flujo normal y pueden ser movidos hacia cualquiera de sus lados.
- **Posicionamiento absoluto.** Los elementos son removidos completamente del flujo normal y asignados a una posición con respecto a un contenedor de bloque.

Esquemas de posicionamiento

➤ Esquema de flujo normal

Para este modelo el atributo **position** puede asumir:

- > **static** (*defecto*). Los elementos respetan el flujo normal. Las propiedades **top**, **right**, **bottom** y **left** no aplican.
- > **relative**. La posición del elemento se calcula en base al flujo normal.
Luego, puede ser desplazado mediante las propiedades **top**, **right**, **bottom** y **left**.

Si se desplaza, los elementos adyacentes no son afectados.

También permite a los hijos usar el posicionado absoluto.

Esquemas de posicionamiento

➤ Esquema flotante

Para este modelo el atributo `float` puede asumir:

- > `left`. El elemento genera una caja de bloque que flota hacia la izquierda. El contenido que habia en esa posición se desplaza hacia la derecha.
- > `right`. El elemento flota hacia la derecha, y el contenido desplazado lo hace hacia la izquierda.
- > `none`. La caja no flota.

Esquemas de posicionamiento

➤ Esquema flotante – Elementos adyacentes

La propiedad **clear** permite indicar si un elemento admite o no cajas adyacentes. Puede asumir los valores:

- > **left**. No admite adyacentes hacia la izquierda.
- > **right**. No admite adyacentes hacia la derecha.
- > **both**. No admite adyacentes en ninguno de sus lados.
- > **none**. No restringe a los adyacentes.

Esquemas de posicionamiento

➤ Esquema de posicionamiento absoluto

Para este modelo el atributo **position** puede asumir:

- > **absolute**. Los elementos salen del flujo normal y su posición se especifica mediante **top**, **right**, **bottom** y/o **left**. Su posición se determina a partir de su ancestro con posicionamiento relativo más cercano.
- > **fixed**. El posicionamiento de la caja se calcula igual que **absolute**, pero con la diferencia que el elemento se mantiene fijo en pantalla.

AGENDA

- ✓ Propiedades de las cajas
- ✓ Tipos de cajas
- ✓ Esquemas de posicionamiento
- ✓ **Flexbox**
- ✓ Bibliografía

Flexbox

➤ ¿Qué es flexbox?

- Representa un modelo de diseño *unidimensional*, el cual permite distribuir los elementos de una página para que se comporten de forma adaptable al dispositivo.
- Representa un modelo de cajas flexibles que mejora las capacidades de alineación de los elementos.

Flexbox

➤ Características

- Produce una mejora sobre el modelo de bloque porque no utiliza la propiedad ***float***, ni hace que los márgenes del contenedor flexible interfieran con los márgenes de sus contenidos.
- Los elementos hijos de una caja flexible pueden colocarse en cualquier dirección y pueden tener dimensiones flexibles para adaptarse al espacio flexible.

Flexbox

➤ Características

- Los diseños complejos se maquetan más fácil y con código más limpio, ya que el orden de visualización de los elementos es independiente del orden que tengan en el código fuente.
- El algoritmo de modelo de diseño de cajas flexibles, no parte de ninguna dirección predeterminada, al contrario del modelo de bloque y de línea.

Flexbox

➤ Características

- El modelo de cajas flexibles es más apropiado para diseños de pequeña escala, mientras que el modelo **grid** es adecuado para diseños de gran escala.
- Es un modelado unidimensional, en contraste con el modelo bidimensional **Grid Layout**, el cual controla columnas y filas a la vez.

Flexbox

➤ ¿Cómo funciona?

- Funciona en término de dos ejes, un eje principal (***main axis***) y un eje cruzado (***cross axis***).
- El eje principal está definido por la propiedad ***flex-direction***, y el eje cruzado es perpendicular a este.

Flexbox

➤ El contenedor FLEX

- Para aplicar el concepto de cajas flexibles, se extiende la definición de la propiedad `display`, adicionando alguno de los valores:
 - `flex` ➔ El elemento se convierte en un contenedor flexible y se comporta como elemento de bloque en el flujo de diseño.
 - `inline-flex` ➔ El elemento se convierte en un contenedor flexible y se comporta como elemento de línea en el flujo de diseño.

Flexbox

➤ Items flexibles

- Los hijos directos de un contenedor flexible se convierten en ítems flexibles. Esto implica que para ellos:
 - Las propiedades `float` y `clear` no aplican
 - `vertical-align` no tiene efecto
 - Los textos que no pertenezcan a un elemento, son envueltos en contenedores de bloques anónimos y son tratados como ítems flexibles.
 - Los márgenes de los ítems adyacentes no se colapsan

Flexbox

➤ Items flexibles

- Por defecto, los ítems flexibles tendrán el siguiente comportamiento inicial:
 - Se despliegan sobre una fila (por defecto `flex-direction` es `row`)
 - Los ítems empiezan desde el margen inicial sobre el eje principal
 - Los ítems no se ajustan sobre el eje principal, pero se pueden contraer
 - Los ítems se ajustarán para rellenar el eje cruzado

Flexbox

➤ Items flexibles

```
<div class="contenedor-flex">
  <div style="clear: both;">No clear</div>
  <div style="float: left;">No float</div>
  <div>
    texto suelto
    <span>texto en span</span>
    <span style="display: block;">span bloque</span>
  </div>
  <div>Item</div>
</div>
```

```
.contenedor-flex{
  display: flex;
}
.contenedor-flex > div{
  border: 1px solid tomato;
}
```

Flexbox

➤ Items flexibles

No clear

No float

texto suelto texto en span
span - bloque

Item

Flexbox

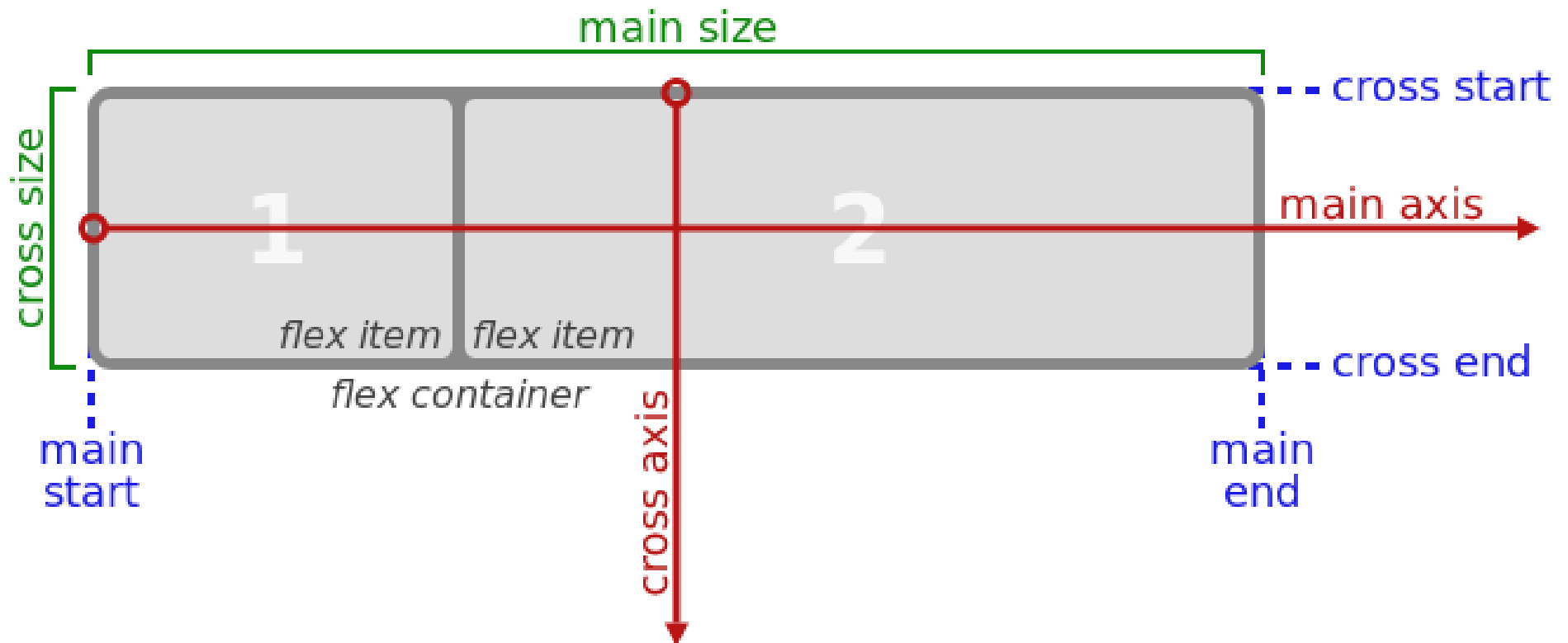
➤ Items flexibles

- El resultado es que todos los ítems se alinearán en una solo fila, usando el tamaño del contenedor como su tamaño en el eje principal.
- Si hay más ítems de los que caben en el contenedor, estos no pasarán más abajo si no que sobrepasarán el margen.
- Si hay ítems más altos que otros, todos los ítems serán ajustados en el eje cruzado para alcanzar al mayor.

Flexbox

➤ Orden y orientación

- El modelo flexible esta basado en un eje principal y otro transversal.



Flexbox

➤ Orden y orientación

- El contenido de un contenedor flexible puede ser orientado en cualquier dirección y orden, a través de las propiedades:
 - `flex-direction` => `row` | `row-reverse` | `column` | `column-reverse`
 - `flex-wrap` => `nowrap` | `wrap` | `wrap-reverse`
 - `order` => `<integer>`

Flexbox

➤ Contenedores flexibles multilíneas

- Si bien flexbox es un modelo unidimensional, es posible lograr que los ítems flex sean repartidos en varias líneas a través de `flex-wrap: wrap`.
- Cuando los ítems sean demasiados para desplegarlos en una línea, serán repartidos en la línea siguiente.
- Al usar `nowrap` los ítems podrían salirse del margen si estos no pudieran contraerse, o no contraerse lo suficiente para ser calzados.

Flexbox

➤ La abreviatura `flex-flow`

- Se pueden combinar las propiedades `flex-direction` y `flex-wrap` en la abreviatura `flex-flow`.
- El primer valor especificado es `flex-direction` y el segundo valor es `flex-wrap`.
 - `flex-flow : row wrap;`

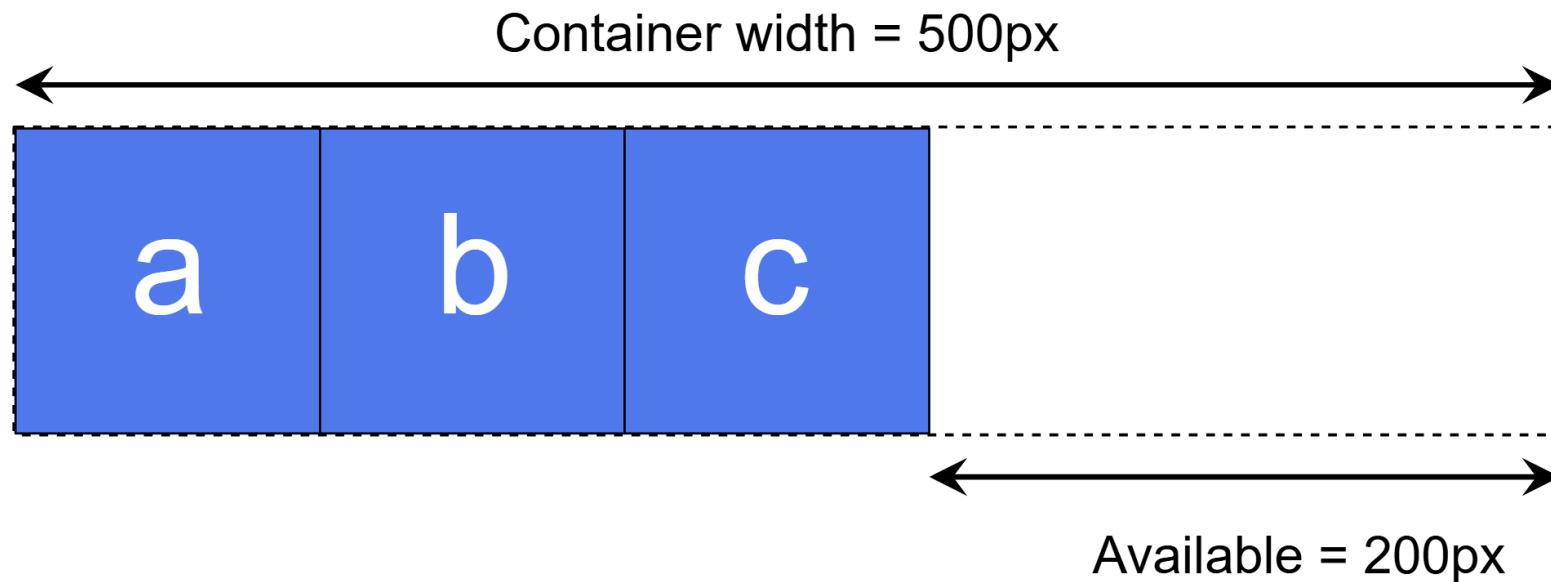
Flexbox

➤ Flexibilidad de los ítems (ejemplo)

- Tenemos tres ítems con un ancho de 100 pixeles cada uno
- El contenedor flexible tiene 500 pixeles de ancho
- Entonces el espacio que se necesita para colocar los ítems es de 300 pixeles
- Esto deja 200 pixeles de espacio disponible
- Si no se cambian los valores iniciales entonces flexbox colocará ese espacio después del último ítem

Flexbox

➤ Flexibilidad de los ítems (ejemplo)



- Si se quisiera rellenar el espacio libre (200px), entonces se necesitan flexibilizar los ítems para llenar el espacio libre.

Flexbox

➤ Flexibilidad de los items

- El aspecto crucial del diseño flexible es la capacidad de flexibilizar los ítems, alterando su ancho y alto para llenar un espacio disponible en el **eje principal**:
 - `flex` : `none` | [`flex-grow` | `flex-shrink` | `flex-basis`]
 - `0 1 auto` ← valor inicial

Flexbox

➤ Flexibilidad de los items

- **flex-grow** → especifica el factor de crecimiento de los ítems. Determina como crecerá el ítem en relación con el resto, cuando el espacio libre es distribuido.
- **flex-shrink** → especifica el factor de contracción. Determina cuanto se encogerá el ítem en relación con el resto, cuando se distribuye el espacio libre negativo.
- **flex-basis** → representa el tamaño inicial del ítem, antes de que el espacio libre se distribuya según los factores *grow* y *shrink*.

Flexbox

➤ Alineación de los ítems – Main axis

- Se pueden alinear los ítems a lo largo del eje principal mediante la siguiente propiedad:
 - `justify-content` → `flex-start` | `flex-end` | `center` | `space-between` | `space-around`
- Aplicable a los contenedores flexibles.

Flexbox

➤ Alineación de los ítems – Cross axis

- Se pueden alinear los ítems a lo largo del eje transversal mediante la siguiente propiedad:
 - `align-items` → `flex-start` | `flex-end` | `center` | `baseline` | `stretch`
- Aplicable a los contenedores flexibles.

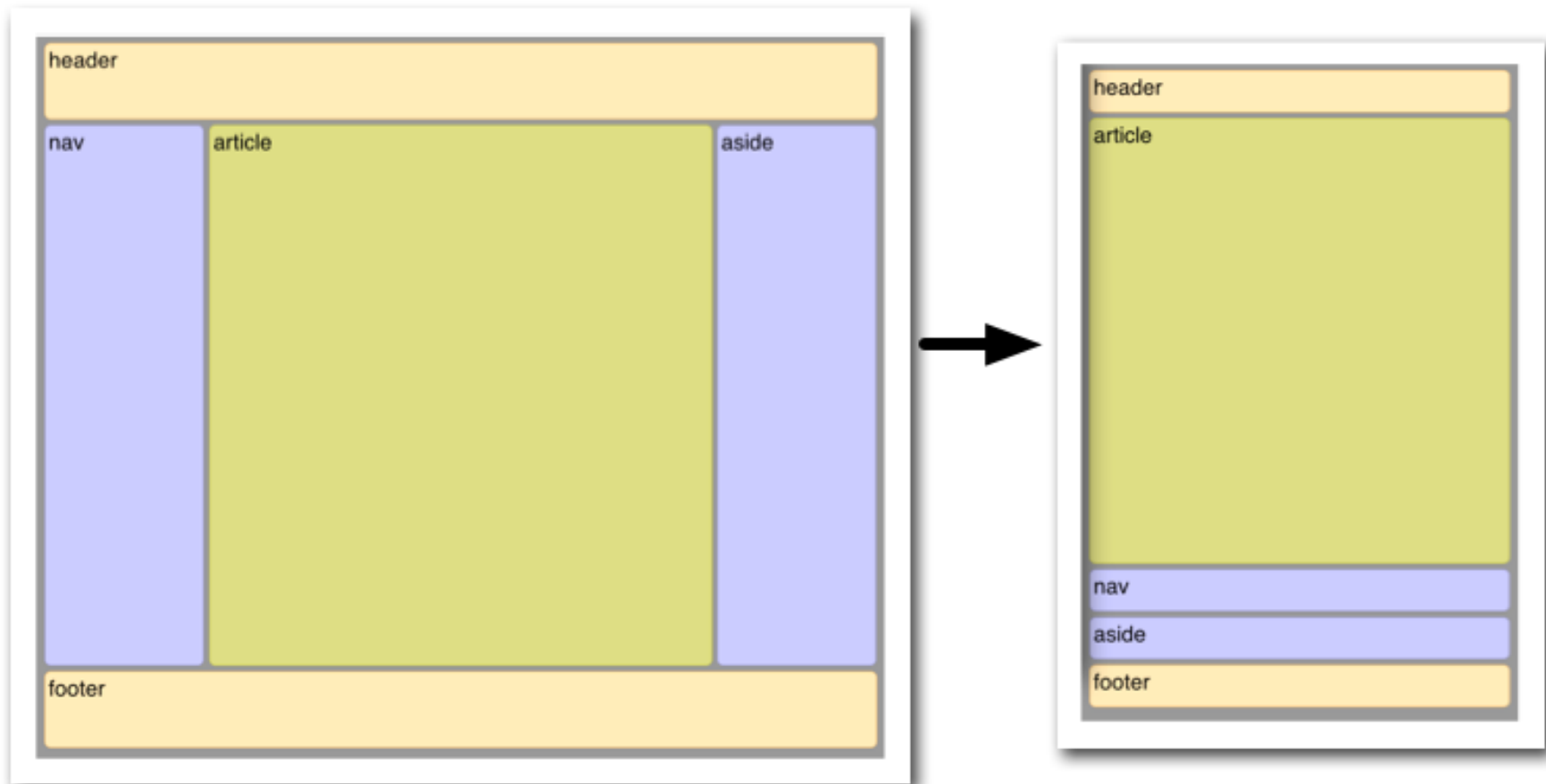
Flexbox

➤ Alineación de las líneas – Cross axis

- Cuando hay múltiples líneas en el contenedor, se pueden alinear los ítems a lo largo del eje transversal mediante la siguiente propiedad:
 - `align-content` → `flex-start` | `flex-end` | `center` | `space-between` | `space-around` | `stretch`
- Aplicable a los contenedores flexibles.
- Se usa cuando ***display: flex*** con ***flex-wrap: wrap*** (o ***wrap-reverse***).
- Y los ítems generan múltiples líneas (porque no caben en una sola).

Flexbox

➤ EJERCICIO: Maquetar la siguiente imagen



AGENDA

- ✓ Propiedades de las cajas
- ✓ Tipos de cajas
- ✓ Esquemas de posicionamiento
- ✓ Flexbox
- ✓ **Bibliografía**

Bibliografía

➤ Sitio de la W3C

- <https://www.w3.org/TR/2020/CR-css-box-3-20201222/>
- <https://www.w3.org/TR/css-display-3/>
- <https://www.w3.org/TR/css-sizing-3/>

➤ Documentación de Mozilla

- <https://developer.mozilla.org/es/>

➤ Diseño Web con CSS

- Ralph G. Schulz. ISBN: 978-84-267-1470-1